TABLE OF CONTENTS

1.0 INTRODUCTION 2

2.0 PROJECT RATIONALE 3

2.1 (a) Background and Sierra Leone Transport Context 3

2.2 (b) Problem Description and Operational Bottlenecks 3

2.3 (c) Community Impact and Primary Beneficiaries 3

2.4 (d) Proposed Solution and SDG 11 Alignment 3

2.5 (e) Computational Thinking Process 3

2.6 (f) Abstraction and Logical Decision Rules 4

2.7 (g) Data Storage and Variable Declarations 4

2.8 (h) Open Source Justification 4

2.9 (i) Digital Public Goods Justification 4

3.0 SYSTEM PSEUDOCODE AND LOGIC DESIGN 5

3.1 Input Process Output (IPO) Architecture 5

3.2 Formal Pseudocode Implementation 5

3.3 Step-by-Step Algorithm 7

4.0 SYSTEM FLOWCHART DIAGRAM 8

4.1 Visual Flowchart Diagram 8

4.2 Flowchart Step Logic and Validation Loops 9

5.0 EXPECTED OUTPUTS AND TEST SCENARIOS 10

5.1 Sample Test Scenarios 10

5.2 Basis for Fare Calculation and Peak-Hour Surcharge 11

6.0 GITHUB REPOSITORY AND VERSION CONTROL EVIDENCE 12

6.1 Repository Setup and URL 12

6.2 Commit History and README Evidence 12

7.0 ABSTRACT 12

8.0 CONCLUSION 13

9.0 REFERENCES 14

1.0 INTRODUCTION

Public transportation across urban centers in Sierra Leone, most notably within the capital city of Freetown, serves as the absolute backbone of daily economic and social life. Every single day, tens of thousands of working citizens, market traders, small business owners, and university students depend heavily on informal commercial transportation networks. These networks consist primarily of minibuses, commonly known as poda-podas, and shared taxicabs that navigate busy arterial roads to connect residential communities with bustling commercial markets and educational institutions.

Despite playing such a critical role in keeping the country's workforce moving, this entire informal transport sector continues to operate entirely through manual methods and traditional human oversight. There are no automated ticketing kiosks, digital passenger tracking systems, or standardized electronic fare collection mechanisms in place. Because operations rely purely on cash exchanges and verbal agreements between drivers, conductors, and passengers, commuters frequently face unpredictable challenges. These include arbitrary fare spikes during rush hours, severe vehicle overcrowding that compromises passenger safety, and a general lack of transparent pricing across different travel distances.

To address these recurring operational bottlenecks and introduce structured logic to local public transit, this project presents a complete software-free logical design for an automated Commuter Transit System. Built as part of the requirements for introductory programming logic and design coursework, the proposed system is engineered to handle vehicle initialization, route selection, passenger capacity tracking, strict input validation, and dynamic fare calculations. By establishing a clear, algorithmic foundation for transport management, this project demonstrates how foundational programming concepts can be effectively applied to solve real world logistical challenges within urban communities in Sierra Leone.

2.0 PROJECT RATIONALE

2.1 (a) Background and Sierra Leone Transport Context

Public transport in Sierra Leone operates through casual arrangements between vehicle owners, transport unions, and park marshals. Since everything is done by hand, driver conductors collect fares through direct bargaining. This means prices change depending on rain, traffic jams, or time of day.

2.2 (b) Problem Description and Operational Bottlenecks

Commuters in Freetown deal with three main problems every day:

1. **Unregulated rush hour price surges where conductors multiply prices during peak traffic hours without official approval.**
2. **Dangerous vehicle overcrowding where vehicles carry far more people than their actual seat limits.**
3. **Unclear pricing where fares change randomly even for short distances because distance zones are not fixed.**

2.3 (c) Community Impact and Primary Beneficiaries

These transit problems hurt low income families and college students who spend most of their daily money just trying to get to school or work. The people who benefit from this system include regular passengers who get clear fares, transport drivers who get automated passenger counting, and transport union officers who get clear rules to enforce.

2.4 (d) Proposed Solution and SDG 11 Alignment

The proposed design calculates fares based on set distance zones and peak hour rules while keeping track of maximum passenger seating limits. This layout directly supports United Nations Sustainable Development Goal 11 for Sustainable Cities and Communities by encouraging safe, cheap, and accessible public transit.

2.5 (e) Computational Thinking and Problem Decomposition

To design a working logic system, the transport problem was broken down into four smaller parts:

- **Part 1: Vehicle setup and capacity check to read maximum seat limits and current passenger numbers.**
- **Part 2: Route and zone input validation to make sure route names and distance numbers fit legal choices.**
- **Part 3: Fare calculation to find the main zone price and add peak hour increases when necessary.**
- **Part 4: Boarding and receipt output to add to the passenger total and print the digital ticket.**

2.6 (f) Abstraction and Logical Decision Rules

Abstraction was used by ignoring physical equipment like printer brands or coin slots. Instead, the focus was kept entirely on decision rules, condition checks, and loops that guide how the software thinks. By stripping away these unnecessary physical complexities, the system establishes clear boundaries for data processing. This is reinforced through strict logical decision rules such as validating distance zones from 1 to 3 and applying dynamic peak-hour surcharges,which ensure the program evaluates inputs accurately and maintains reliable execution from passengers.

2.7 (g) Data Storage and Variable Declarations (Flowgorithm Standard)

To follow proper logic guidelines and work correctly in Flowgorithm, data must be stored inside memory variables before it gets used. The system clearly sets up these specific variables:

- **routeID (String): Stores route code like FRT-01.**
- **maxCapacity (Integer): Maximum legal seats in the vehicle.**
- **passengerCount (Integer): Current number of passengers inside.**
- **distanceZone (Integer): Distance zone where 1 is short, 2 is medium, and 3 is long.**
- **peakHour (String): Checks rush hour status using YES or NO.**
- **baseFare (Real): Initial fare calculated for that distance.**
- **finalFare (Real): Final ticket price after adding peak hour extras.**
- **boardMore (String): Loop flag to check if more people want to board.**

2.8 (h) Open Source Software (OSS) Justification

Choosing an open-source approach for this commuter transit system is essential because it eliminates heavy financial barriers and vendor lock-in that often cripple tech initiatives in developing countries like Sierra Leone. Proprietary software solutions require expensive recurring subscription fees, closed licenses, and specialized external consultants just to make basic updates or fix bugs. By building this system on open-source principles, local transport unions, municipal authorities, and university coding students can freely access, inspect, modify, and distribute the source code without paying prohibitive costs. Furthermore, an open-source model invites community-driven collaboration, allowing student developers and local tech enthusiasts to continually improve the codebase, patch security flaws, and tailor the system to fit the unique realities of Freetown's bustling transport networks

2.9 (i) Digital Public Goods (DPG) Framework Alignment

The digital public goods framework emphasizes open-source software, open data, open standards, and privacy protection that help attain the Sustainable Development Goals without locking communities into commercial monopolies. This commuter transit system aligns naturally with DPG core principles because it is designed to be universally accessible, transparent, and non-commercial, serving the public interest of everyday commuters and transport workers. By operating on open logic and remaining free from proprietary restrictions, the system empowers local public institutions to adopt, scale, and integrate the technology into broader smart-city infrastructures. This ensures that technological advancements in Sierra Leone's transport sector remain ethical, locally controlled, and directly beneficial to the public rather than driven by private corporate profit motives.

3.0 SYSTEM PSEUDOCODE AND LOGIC DESIGN

3.1 Input Process Output (IPO) Architecture

- **Input: routeID, maxCapacity, passengerCount, distanceZone, and peakHour.**
- **Process: Check capacity limits, verify distance zone choice between 1 and 3, compute base fare, apply a 1.25 multiplier if peak hour equals YES, and update passenger count.**
- **Output: Printed fare receipt, updated passenger count, and capacity limit alerts.**

3.2 Formal Pseudocode Implementation

```
Begin CommuterTransitSystem

   // Variable Declarations

   Declare String routeID, peakHour, boardMore

   Declare Integer distanceZone, passengerCount, maxCapacity

   Declare Real baseFare, finalFare

   // Initial State Setup

   boardMore = "YES"

   Output "=== FREETOWN COMMUTER TRANSIT SYSTEM ==="

   Output "Enter Route ID:"

   Input routeID

   Output "Enter Vehicle Maximum Capacity:"

   Input maxCapacity

   Output "Enter Initial Passenger Count:"

   Input passengerCount

   // Main Passenger Boarding and Capacity Enforcement Loop

   While boardMore == "YES" AND passengerCount < maxCapacity Do

      Output "Enter Distance Zone (1: Short, 2: Medium, 3: Long):"

      Input distanceZone

      // Input Validation Loop for Distance Zone

      While distanceZone < 1 OR distanceZone > 3 Do

         Output "INVALID ZONE! Enter 1, 2, or 3:"

         Input distanceZone
```

```
End While
Output "Is this Peak Rush Hour? (YES/NO):"
Input peakHour

// Base Fare Determination Based on Tiered Zones
If distanceZone == 1 Then
   baseFare = 5.00
Else If distanceZone == 2 Then
   baseFare = 10.00
Else
   baseFare = 15.00
End If
// Surcharge Calculation for Peak Hours
If peakHour == "YES" Then
   finalFare = baseFare * 1.25
Else
   finalFare = baseFare
End If
// Increment Passenger State
passengerCount = passengerCount + 1
// Boarding Summary Output
Output "BOARDING SUCCESSFUL!"
Output "Route: ", routeID
Output "Fare Due: Le ", finalFare
Output "Current Passengers: ", passengerCount, " / ", maxCapacity
// Capacity Threshold Evaluation
If passengerCount >= maxCapacity Then
   boardMore = "NO"
   Output "ALERT: Vehicle is FULL. Boarding stopped."
```

Else

Output "Board another passenger? (YES/NO):"

Input boardMore

End If

End While

Output "Session finished. Have a safe journey!"

End CommuterTransitSystem.

3.3 (Step-by-Step Algorithm)

System Initialization: Prompt the operator to enter `routeID`, legal vehicle `maxCapacity`, and starting `passengerCount`.

Conditional Entry Check: Check if `boardMore == "YES"` AND `passengerCount < maxCapacity`. If false, proceed directly to Step 8.

Distance Zone Input & Validation: Prompt for `distanceZone`. If `distanceZone` is less than 1 or greater than 3, loop repeatedly with an error message until a valid integer (1, 2, or 3) is provided.

Peak Hour Input: Prompt the operator for `peakHour` status ("YES" or "NO").

Base Pricing Determination:

- **If `distanceZone == 1`, set `baseFare = 5.00`.**
- **If `distanceZone == 2`, set `baseFare = 10.00`.**
- **If `distanceZone == 3`, set `baseFare = 15.00`.**

Peak Surcharge Evaluation: If `peakHour == "YES"`, calculate `finalFare = baseFare * 1.25`. Otherwise, set `finalFare = baseFare`.

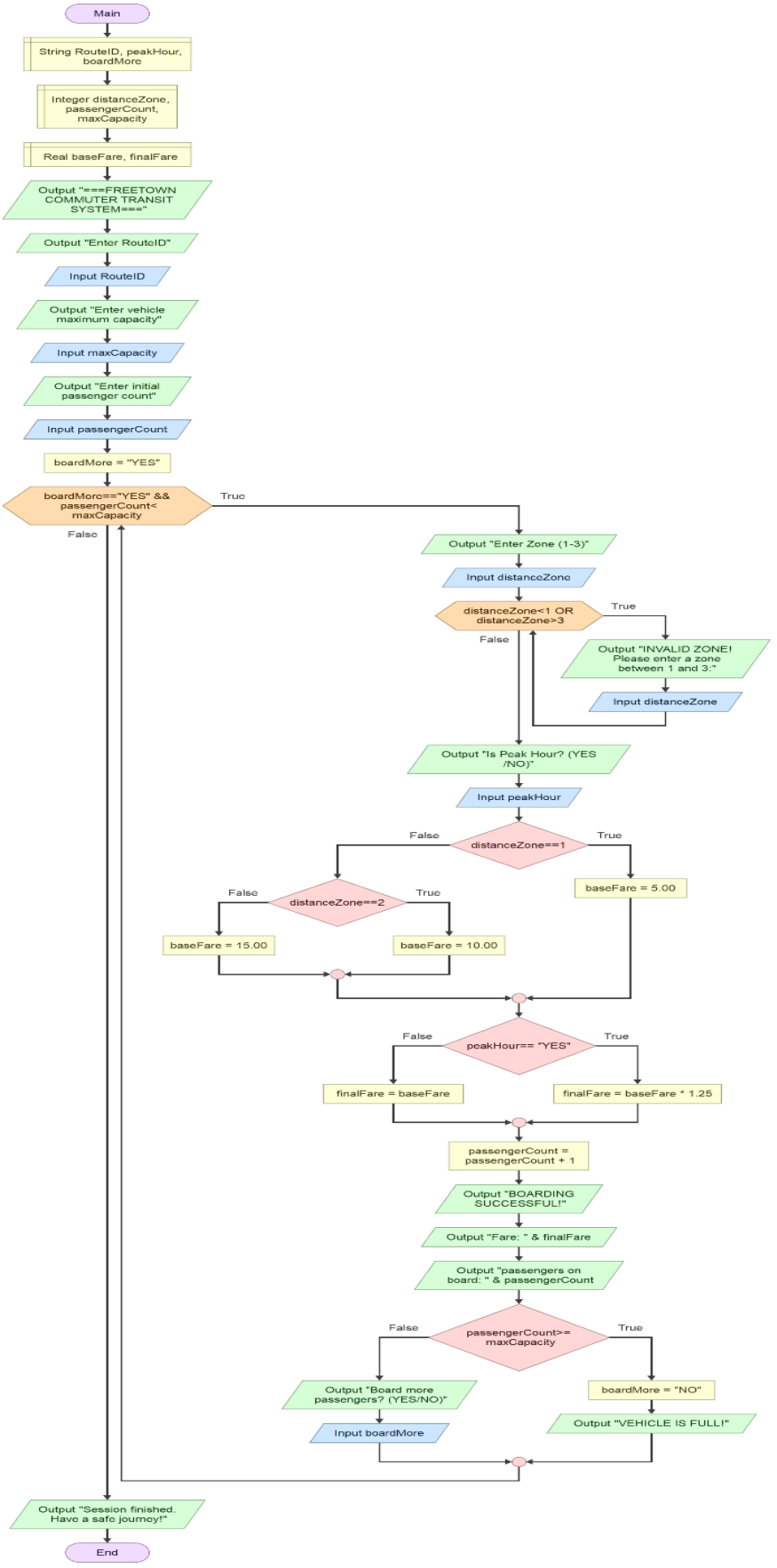
Boarding Update & Capacity Enforcement:

- **Increment `passengerCount = passengerCount + 1`.**
- **Display ticket details including `routeID`, `finalFare`, and updated passenger load (`passengerCount / maxCapacity`).**
- **If `passengerCount >= maxCapacity`, set `boardMore = "NO"` and display `"ALERT: Vehicle is FULL. Boarding stopped."`**
- **Otherwise, prompt for `boardMore` ("YES" or "NO") and repeat from Step 2.**

Program Termination: Display `"Session finished. Have a safe journey!"` and terminate execution.

4.0 SYSTEM FLOWCHART DIAGRAM

4.1 Visual Flowchart Diagram



4.2 Flowchart Step Logic and Validation Loops

- **Start oval sets up variable slots in memory.**
- **Input parallelograms ask the user for route codes, max vehicle seats, and starting passenger counts.**
- **Decision diamonds check if the distance zone is valid between 1 and 3, and whether the passenger count has reached the maximum capacity.**
- **Process rectangles run the fare calculation and update the passenger count by one for every successful ticket.**

5.0 (Sample Test Scenarios & Fare Rationale)

5.1 Sample Test Scenarios

Test Case	Scenario Description	Inputs	Expected Output	System Outcome
Case 1	Standard Off-Peak Trip (Short Distance)	Route: FRT-01 Max Cap: 15 Initial Load: 0 Zone: 1 Peak: NO	Base Fare: Le 5.00 Final Fare: Le 5.00 Passengers: 1 / 15	Boarding successful; system prompts to board another passenger.
Case 2	Peak-Hour Long Distance Trip	Route: FRT-02 Max Cap: 10 Initial Load: 5 Zone: 3 Peak: YES	Base Fare: Le 15.00 Peak Multiplier: 1.25 Final Fare: Le 18.75 Passengers: 6 / 10	Peak surcharge correctly applied (+25%); boarding allowed to continue.
Case 3	Input Error Handling (Invalid Zone)	Zone Input: 5	Output: "INVALID ZONE! Enter 1, 2, or 3:"	User trapped in validation loop until entering valid zone (1, 2, or 3).

Case 4	Immediate Vehicle Capacity Threshold	Max Cap: 14 Initial Load: 13 1 Passenger Boarded	Output: "ALERT: Vehicle is FULL. Boarding stopped."	boardMore set to "NO"; main While loop terminates immediately.

5.2 Basis for Fare Calculation and Peak-Hour Surcharge

- **Distance-Based Fare Tiers:** The pricing structure divides commuter routes in Freetown into three distinct operational zones:
  - **Zone 1 (Short Distance - Le 5.00):** Covers short-distance intra-city shuttle trips (0–5 km).
  - **Zone 2 (Medium Distance - Le 10.00):** Covers intermediate routes connecting residential sub-districts to central hubs (5–15 km).
  - **Zone 3 (Long Distance - Le 15.00):** Covers major cross-town arterial corridors exceeding 15 km.
- **25% Peak-Hour Surcharge Rationale:** During morning and evening rush hours, severe traffic congestion significantly increases fuel consumption and vehicle wear-and-tear while reducing the total number of round trips a vehicle can complete per shift. The 25% multiplier (1.25) compensates operators for operating delays while regulating peak demand across Freetown's transport networks.

6.06.0 GITHUB REPOSITORY AND VERSION CONTROL EVIDENCE

6.1 Repository Setup and URL

- **Repository Name:** `PROG101_Assignment1_RugiatuMariaKamara_905006520`
- **Repository URL:**
- **Visibility: Public**
- **Files in Repository:**
  - `README.md` **(Project overview and instructions)**
  - **Final report**
  - `pseudocode.txt`
  - **Flowchart diagram**

6.2 Commit History and README Evidence

The repository was managed through regular, iterative version control commits, documenting the project's progress from initial logic setup to final documentation and report compilation.

Figure 8: Git commit history and version control logs

Figure 9: Repository file structure and documentation setup

Once you drop this into Section 6.0, add your two screenshots under Figures 8 and 9, and ensure your Table of Contents reflects it, your entire assignment package will be fully complete! How does this look to you?

7.0 CONCLUSION

This assignment demonstrates how computational thinking, structured programming logic, and algorithm design can be practically applied to resolve complex, everyday public transit challenges in Sierra Leone. By examining the informal transport sector across Freetown, specifically the operational habits of minibuses (poda-podas) and shared taxis, this project identified key systemic issues including unpredictable price surges during peak rush hours, dangerous passenger overcrowding, and a lack of standardized zone

pricing.

Through problem decomposition and abstraction, these real world transit bottlenecks were translated into a clear, software-independent logical model. The design strictly incorporates explicit variable data storage, robust input validation loops, and dynamic fare calculation functions. Enforcing capacity boundaries through conditional While loops ensures that vehicles cannot accept additional passengers once their legal limit is reached, directly addressing safety concerns on local roads. Furthermore, establishing set distance zones paired with structured peak-hour multipliers removes price ambiguity for daily commuters, university students, and low-income families.

Documenting and publishing this system on GitHub using Open Source standards and Digital Public Goods frameworks guarantees that the design remains accessible, transparent, and adaptable for local developers, transport unions, and public authorities. Ultimately, this project proves that logical system design serves as a vital first step toward modernizing urban infrastructure, aligning directly with United Nations Sustainable Development Goal 11 to foster safe, fair, and sustainable community transportation systems.

8.0 REFERENCES

1. **Digital Public Goods Alliance. (2025). *Standard for Digital Public Goods*. DPGA Publications.**
2. **Farrell, J. (2018). *Programming Logic and Design, Comprehensive* (9th ed.). Cengage Learning.**
3. **Fullah, E. (2026). *PROG101 Principles of Programming Logic and Design: Individual Assignment Brief*. Department of Information and Communication Technology, Limkokwing University of Creative Technology, Sierra Leone.**
4. **Open Source Initiative. (2026). *The Open Source Definition*. Available at: https://opensource.org/osd**
5. **United Nations. (2015). *Transforming our world: The 2030 Agenda for Sustainable Development (Goal 11: Sustainable Cities and Communities)*. UN Publishing.**